

Whose Memory Can Be Read? Cross-Body Readability of Persistent Action Memory for Frozen VLA Policies

Anonymous Author(s)

Fig. 1. [Fig. 1 teaser — two panels only].

Abstract—A robot that improves during deployment must put what it learned somewhere. Retrieval-augmented frozen vision-language-action (VLA) policies already accumulate deployment experience without gradient updates, and cross-embodiment retrieval pools already exist. What has not been shown is a persistent store of *executable action content*, written by a robot *during its own deployment*, that a *second deployed robot* reads with *no further update to its policy*. We ask what an entry must contain for this to be possible. Each experience is written at four levels of abstraction—a re-groundable language summary, an object-centric subgoal sequence, object-anchored end-effector deltas, and a joint-space action chunk—under a body-agnostic key with a learned alignment map, and every level is decoded into the reader’s own action space before it is injected into the reader’s frozen flow-matching sampler. A fixed-base Franka FR3 and a mobile manipulator both write to and read from one store, in both directions. We report the transferability spectrum across the four levels, decompose the cross-body loss into key-side, retargeting and value-side terms, and report the per-trial memory-harm rate and the injection rate that make these numbers interpretable. Admitting a third, previously unseen reader costs [N] paired episodes and [M] GPU-minutes, and no policy gradient.

I. INTRODUCTION

[P1: phenomenon + why co-training is not the answer]

[P2: the premise — persistent, self-written, action-bearing memory is body-locked]

[P3: transferability is decided when the entry is written]

[P4: layered value + body-agnostic key + one shared readout interface]

[P5: what is measured, the four predictions, and the honest boundary]

Contributions.

- 1) **We quantify what a reader swap costs a retrieval memory, in threshold-free terms.** We measure retrieval quality for same-body and cross-body queries under four key constructions, reporting precision@ k and ROC-AUC against a positive-pair definition derived from our own (task, phase) annotation, and reporting the recall—equivalently, the abstention—that a cross-body reader must pay to reach the same precision as a same-body reader. We deliberately avoid arguing from the default similarity gate of any single system: the gate value of a state-of-the-art same-body retriever is a per-deployment trade-off knob, not a property of

the representation, and its own sensitivity study shows the method is nearly flat across it.

- 2) **We make the transferability of an experience an experimentally separable variable.** One collection produces four value levels per entry under a shared key and, crucially, a shared readout interface: every level is decoded into the reader’s action space and injected at the same point of the same frozen sampler, so that abstraction level is varied while the injection path is held fixed. We report each level with oracle content and with automatically generated content, separating what the representation costs from what the content generator costs, and we compare against a derive-on-read store that materialises only the lowest level and reconstructs the rest at read time.
- 3) **We report a persistent action store that two deployed bodies share, write and read bidirectionally, with no gradient reaching either policy once deployed.** Each body’s base policy is fine-tuned once, before deployment, on tasks disjoint from the memory-evaluation tasks; nothing is updated when the store is admitted or when it grows. We further admit a third, previously unseen reader body and report its admission cost in paired episodes and GPU-minutes, alongside the cost of a per-body target-side fine-tune on the same pair. We report the per-trial memory-harm rate—the fraction of paired trials, at matched initial state and matched sampler noise, in which the store makes the reader worse than its memoryless base—together with the injection rate that conditions it.
- 4) **We decompose the cross-body loss and characterise a shared store.** An oracle ladder attributes the gap between a same-body and a cross-body reader to three additive terms—retrieval, retargeting, and value inexecutability—without post-hoc human labelling of failures. On a corpus-seeded store of [1e4–1e5] entries ([N] episodes) shared across tasks and bodies, we show that the binding constraint is retrieval precision and cross-body interference rather than capacity.

We explicitly do not claim the intermediate-state injection into a flow-matching sampler, the confidence-adaptive guidance strength, the abstain-on-low-confidence fallback, or the component-aware aggregation of retrieved chunks—all adopted from [1] and used unchanged once an entry has been decoded into the reader’s action space. Nor do we claim gradient-free growth of a retrieval pool, demonstrated by [2], nor entry-level revocability, demonstrated by [3].

TABLE I

WHERE A DEPLOYMENT EXPERIENCE GOES, AND WHO CAN READ IT AFTERWARDS. [FILL EACH CELL FROM THE FIRST-HAND PDF AUDIT IN IDEA/THREAT_RAM_ROBOOS.MD; MARK UNKNOWN AS “?”, NEVER GUESS.] COLUMNS: *Pers.* PERSISTS ACROSS EPISODES; *Act.* THE ENTRY CARRIES EXECUTABLE ACTION CONTENT; *Self* WRITTEN BY A ROBOT DURING ITS OWN DEPLOYMENT; *X-read* READABLE BY A BODY THAT DID NOT WRITE IT; *Del.* ENTRY-LEVEL DELETION.

Memory	Pers.	Act.	Self	X-read	Del.
Episodic buffers [4], [5]	×	✓	✓	—	—
In-context [6], [7]	×	✓	×	✓	—
Adapters [3], [8]	✓	✓	✓	×	✓
Retrieval pool [2]	✓	✓	×	[?]	✓
Affordance memory [9], [10]	✓	[?]	×	✓	✓
Replay buffers [11]–[13]	✓	✓	×	×	✓
Success memory [1], [14], [15]	✓	✓	✓	×	✓
Scene/state memory [16]	✓	×	✓	✓	✓
PortMem (ours)	✓	✓	✓ [†]	✓	✓

[†] for the deployment store $\mathcal{S}_{\text{deploy}}$; the scaling study uses a corpus-seeded store $\mathcal{S}_{\text{corpus}}$.

II. RELATED WORK

A. Memory That Ends With the Episode

[RW-A]

B. Memory Written Into Parameters

[RW-B]

C. Memory Written As Explicit Entries

[RW-C]

III. METHOD

A. Problem Setting

We call the body that produced an entry its *writer* and the body that consumes it a *reader*; a *reader swap* is the substitution of one for the other with the store held fixed. Unlike [2], where the pool embodiment is never deployed, both of our bodies are deployed and each is both a writer and a reader.

Let $\mathcal{B} = \{b_A, b_B\}$ be two bodies with frozen flow-matching VLA policies $\pi_b(\mathbf{a} \mid \mathbf{o}, \ell)$ emitting chunks $\mathbf{a} \in \mathbb{R}^{H \times d_b}$. Each π_b is fine-tuned once, before deployment, on a task set $\mathcal{T}_{\text{base}}$ with $\mathcal{T}_{\text{base}} \cap \mathcal{T}_{\text{eval}} = \emptyset$, and is never updated again (experiment E0). The store is

$$\mathcal{M} = \{e_i\}_{i=1}^N, \quad e_i = (\mathbf{k}_i, \{v_i^{(\ell)}\}_{\ell=1}^4, \beta_i, c_i, t_i), \quad (1)$$

with $\mathbf{k}_i$ a body-agnostic key, $v_i^{(\ell)}$ the value at abstraction level ℓ , β_i the writer-body tag, c_i a write-gate calibration score and t_i a timestamp. Writing is a single step with no gradient,

$$\mathcal{M} \leftarrow \mathcal{M} \cup \{e(\sigma) : \sigma \in \text{Seg}(\tau), g(\tau) = 1\}, \quad (2)$$

where Seg segments a rollout into motion segments and g is the write gate. At read time on body b ,

$$\mathcal{I} = \text{TopK}(\text{sim}(\mathbf{q}_b(\mathbf{o}_t), \mathbf{k}_i)), \quad (3)$$

$$\hat{\mathbf{a}}_i = \mathcal{D}_b^{(\ell)}(v_i^{(\ell)}, \mathbf{o}_t) \in \mathbb{R}^{H \times d_b}, \quad (4)$$

where $\mathcal{D}_b^{(\ell)}$ decodes level ℓ into the *reader’s* action space. This shared interface is what makes abstraction level separable from injection path. The prior and its injection follow [1] unchanged:

$$\mathbf{a}_{\text{elite}} = \sum_{i \in \mathcal{I}} w_i \hat{\mathbf{a}}_i, \quad \mathbf{x}_{t_0} = (1 - t_0) \mathbf{a}_{\text{elite}} + t_0 \epsilon. \quad (5)$$

We report $\text{TR}_\ell = \text{SR}_\ell^{\text{cross}} / \text{SR}_\ell^{\text{same}}$ for context only. A ratio of success rates can be driven to one by abstaining more often; we therefore state every claim in absolute success rate against each reader’s own memoryless base at matched initial state, and report the injection rate alongside.

B. What an Entry Is

An entry is one *motion segment*, delimited by gripper open/close events and by zero-crossings of end-effector speed. Segmentation is a state machine over signals already present in our corpus; no learned segmenter is used. Mean segment length is [L] steps, so a store of [N] entries corresponds to [M] episodes; we report both units throughout.

[Table: four value levels — content, generation, size, prior occupant]

Both the object-centric subgoals and the object-anchored deltas require a 6-DoF object pose. We obtain it with a detector and tracker before grasp, and with the gripper pose itself after grasp, exploiting the rigid attachment; levels (2) and (3) therefore share a failure mode, and we report them as such.

C. A Body-Agnostic Key and the Alignment Map

[Key constructions and the alignment map]

The alignment map is trained on weakly paired data—the two bodies performing the same task, collected independently rather than in correspondence—on a task set $\mathcal{T}_{\text{align}}$ disjoint from the evaluation tasks $\mathcal{T}_{\text{eval}}$. All main-table numbers use the held-out-task protocol; in-domain numbers appear only as an upper bound. The map is the *only* component trained by gradient descent, has fewer than 2M parameters, and trains in [X] GPU-minutes; that figure is the admission cost reported in Sec. V.

D. Readout: One Interface, Four Levels

Every level is decoded into the reader’s action space before injection: level (1) is re-grounded by a frozen VLM in the reader’s current image and emits subgoals, becoming level (2); level (2) is instantiated as object-frame waypoints, becoming level (3); level (3) is transformed into the reader’s task frame and solved to a joint-space chunk by differential IK, becoming level (4); level (4) is used directly. All four therefore enter the same aggregation and the same intermediate-state injection of Eq. (5), so that the level of abstraction is varied while the injection path is held fixed.

For same-body reads the highest-fidelity level (4) is preferred; for cross-body reads the readout climbs from (4) upward until decoding succeeds and the result is kinematically feasible. *Which level it lands on is itself an experimental reading.*

Fig. 2. [Fig. 2 System overview]. A rollout is segmented into motion segments; a write gate admits only progress-verified prefixes, and each admitted segment is stored at four levels of abstraction with a body-agnostic key and a writer tag. At read time, retrieved entries at any level are decoded into the reader’s action space, so that all levels share one injection path. The shaded block is adopted unchanged from [1].

Algorithm 1 Deployment loop for reader body b (policy frozen)

Require: store $\mathcal{M}$, frozen policy π_b , key encoder q_b , alignment map h_b

```

1: for each control step  $t$  at a chunk boundary do
2:    $k_q \leftarrow h_b(q_b(o_t))$ 
3:    $\mathcal{I} \leftarrow \text{TopK}_i \text{sim}(k_q, k_i)$ ; filter by DTW consistency  $\{[1]\}$ 
4:   if  $\mathcal{I} = \emptyset$  then
5:      $a \leftarrow \pi_b(o_t, \ell)$ ; continue {abstain}
6:   end if
7:    $\ell_i \leftarrow$  highest executable level;  $\hat{a}_i \leftarrow \mathcal{D}_b^{(\ell_i)}(v_i, o_t)$ 
8:   drop  $i$  whose retarget/IK fails; abstain if none remain

9:    $a_{\text{elite}} \leftarrow \sum_i w_i \hat{a}_i$ ;  $c \leftarrow \alpha \tilde{s} - \beta \sigma_{\text{DTW}}$ 
10:   $t_0 \leftarrow t_{\min} + (1 - t_{\min}) \sigma(-\gamma c)$ 
11:   $a \leftarrow \text{FlowSample}(\pi_b, o_t, \ell, x_{t_0})$ 
12:  execute  $a$ ; buffer  $(k_q, \text{executed chunk})$ 
13: end for
14: at episode end:  $s \leftarrow \text{WriteGate}(\text{rollout})$ 
15: if  $s \geq \eta$  then
16:    $\mathcal{M} \leftarrow \mathcal{M} \cup \{e(\sigma, \text{tag} = b)\}_{\sigma \in \text{Seg}(\text{prefix})}$ 
17: end if

```

For entries decoded into the reader’s action space, we adopt the aggregation, the intermediate-state initialisation of the flow sampler, the confidence-adaptive guidance strength, and the abstain-and-fall-back rule of [1] unchanged, including its component-aware aggregation of Euclidean, $SO(3)$ and gripper components. What is not part of that method, and what we describe here, is the decoding path that maps entries at different levels of abstraction into a single such action-space prior.

Levels (1)–(3) are deterministic functions of level (4) together with the stored frames, so an information-equivalent store could materialise only (4) and derive the rest at read time. The difference is where the cost is paid: deriving levels (1) and (2) at read time requires a VLM call inside the control loop and adds [X] ms per retrieval against a budget of [Y] ms at the chunk boundary.

E. Write Gate and Lifecycle

[Write gate, decay, capacity bound, entry-level deletion]

Entry-level deletion is a property we share with adapter-based memories [3] rather than a distinguishing one.

IV. EXPERIMENTAL SETUP

[Platforms, task tiers, base policies, store construction]
 [The four disclosures]
 [Reporting protocol]

V. RESULTS

A. The Cost of a Reader Swap

[V-A]

B. The Transferability Spectrum

[V-B]

C. Where the Loss Goes

[V-C]

D. Direction

[V-D]

E. Sharing, Scale, and Harm

[V-E]

F. The Cost of Admitting a Reader

[V-F]

[Table III — real-robot main results: paired Delta vs. each body’s own memoryless base, with injection rate and harm rate in every cell]

[Table IV — ablations: six control arms and the three sharing conditions]

VI. CONCLUSIONS AND LIMITATIONS

[Conclusions and limitations]

REFERENCES

- [1] J. Zhao, H. Yang, Y. Hu, Y. Gao, *et al.*, “Retrieve-then-steer: Online success memory for test-time adaptation of generative VLAs,” *arXiv preprint arXiv:2605.10094*, 2026.
- [2] J. Park, J. Park, T. Kim, S. Choi, *et al.*, “Retrieve, don’t retrain: Extending vision language action models to new tasks at test time,” *arXiv preprint arXiv:2606.15631*, 2026.
- [3] R. Römer, Y. Zhang, Y. Li, and A. P. Schoellig, “CLARE: Continual learning for vision-language-action models via autonomous adapter routing and expansion,” *IEEE Robotics and Automation Letters*, 2026, arXiv:2601.09512.
- [4] H. Shi, B. Xie, Y. Liu, L. Sun, *et al.*, “MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation,” in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2508.19236.
- [5] M. Torne, K. Pertsch, H. Walke, K. Vedder, *et al.*, “MEM: Multi-scale embodied memory for vision language action models,” *arXiv preprint arXiv:2603.03596*, 2026.
- [6] Y. Jiang, Y. Chebotar, R. Zheng, *et al.*, “RoboTTT: Context scaling for robot policies,” *arXiv preprint arXiv:2607.15275*, 2026.
- [7] R. Shah, S. Liu, Q. Wang, *et al.*, “MimicDroid: In-context learning for humanoid robot manipulation from human play videos,” *arXiv preprint arXiv:2509.09769*, 2025.
- [8] S. Si, Y. Lu, R. Yang, Z. Ye, *et al.*, “VLA-Pro: Cross-task procedural memory transfer for vision-language-action models,” *arXiv preprint arXiv:2605.29562*, 2026.
- [9] Y. Kuang, J. Ye, H. Geng, J. Mao, *et al.*, “RAM: Retrieval-based affordance transfer for generalizable zero-shot robotic manipulation,” in *Conference on Robot Learning (CoRL)*, 2024, arXiv:2407.04689.
- [10] Y. Ju, K. Hu, G. Zhang, G. Zhang, *et al.*, “Robo-ABC: Affordance generalization beyond categories via semantic correspondence for robot manipulation,” *arXiv preprint arXiv:2401.07487*, 2024.
- [11] N. D. Palo and E. Johns, “On the effectiveness of retrieval, alignment, and replay in manipulation,” *IEEE Robotics and Automation Letters*, 2024, accepted December 2023; arXiv:2312.12345.
- [12] —, “DINOBot: Robot manipulation via retrieval and alignment with vision foundation models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, arXiv:2402.13181.

- [13] J. Pari, N. M. Shafiullah, S. P. Arunachalam, and L. Pinto, “The surprising effectiveness of representation learning for visual imitation,” *arXiv preprint arXiv:2112.01511*, 2021, tODO: venue NOT verified as of 2026-08-22 — do not label RSS/CoRL without checking.
- [14] S. Wu, Y. Ji, Q. Li, Z. Zhang, *et al.*, “Dejavu: Towards experience feedback learning for embodied intelligence,” *arXiv preprint arXiv:2510.10181*, 2025.
- [15] Z. Li, Z. Li, K. Zhou, and J. Gu, “RoboHarness: A memory-augmented policy harness for vision-language-action model robustness via in-context adaptation,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026, arXiv:2603.24060.
- [16] H. Tan, X. Hao, C. Chi, M. Lin, *et al.*, “RoboOS: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration,” *arXiv preprint arXiv:2505.03673*, 2025.